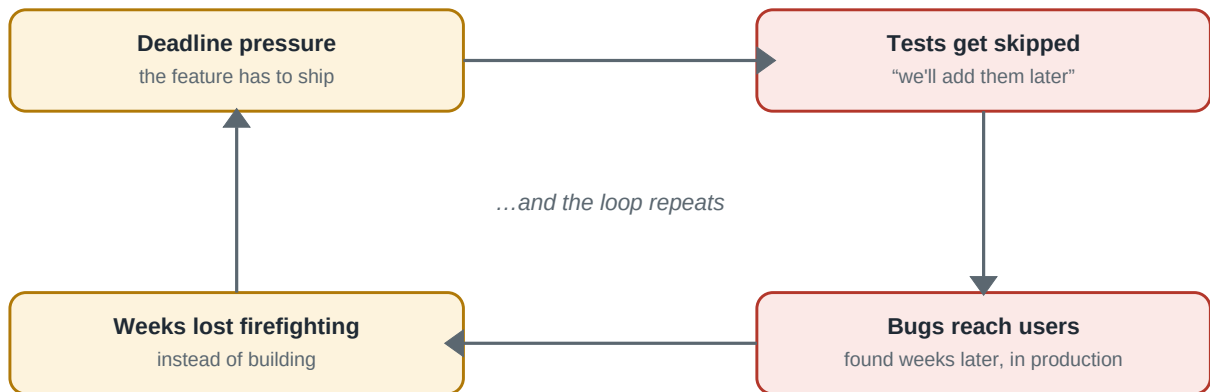


test-automator

A safety net for every code change — built automatically, verified before you see it

The problem — in any team, any language

Unit tests are small automated checks that prove a piece of code still does what it's supposed to. Every team agrees they matter. And yet almost every team under-invests in them, because writing them by hand is slow, repetitive work that competes with the next deadline — and the deadline usually wins:



Skipping tests doesn't remove the cost of bugs — it defers the cost to the worst possible moment. The later a bug is found, the more it costs to fix:

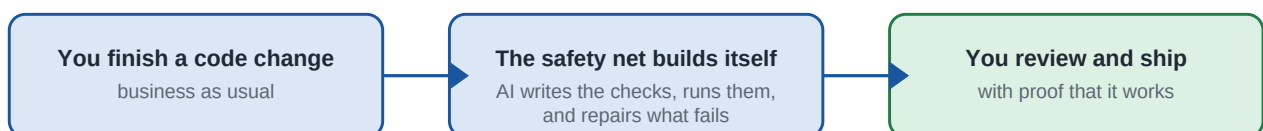
Cost of the same bug, by where it is discovered



Tests move bug discovery to the left of this chart. That's their entire economic argument — and it's why coverage that erodes quietly is a business problem, not just an engineering one.

What test-automator does — in plain words

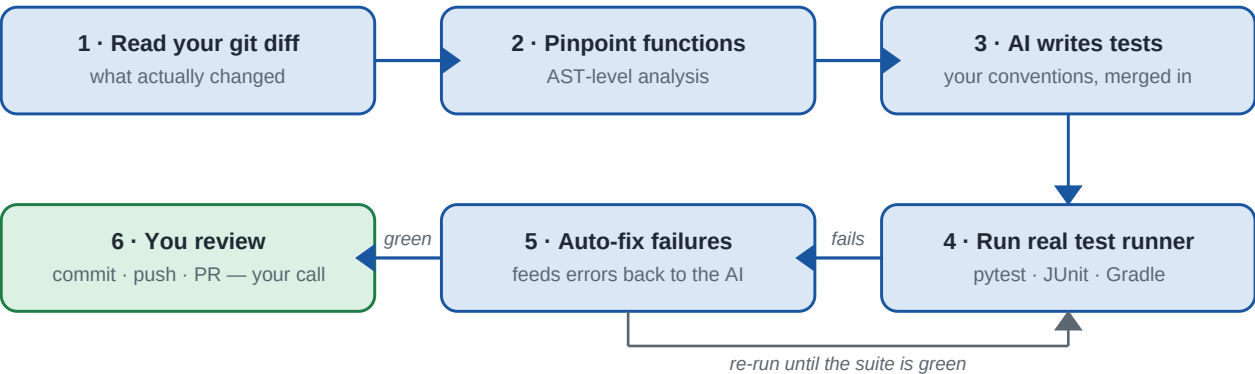
test-automator makes the safety net build itself. When an engineer finishes a change, one command examines exactly what changed, asks the AI to write the matching checks in the team's own style, runs them for real, and repairs anything that doesn't pass — before a human ever looks at them:



The engineer stays in charge: generated tests are presented for review, never silently added. What was hours of manual work becomes minutes of review.

Under the hood — for the engineers

Technically, it's a six-step pipeline that turns a **git diff** into **passing, convention-following unit tests**:



The fix loop is bounded and honest: environment problems — missing dependencies, JDK mismatches, build-cache locks — are detected and reported with actionable messages instead of being “fixed” blindly.

Works with your stack

Languages	Python (pytest) • Kotlin (JUnit 5 + MockK + Strikt) • Java (JUnit + Mockito)
AI backends	Claude Code (default) • GitHub Copilot CLI • Gemini CLI • any custom CLI
Build systems	pytest projects, Gradle projects (wrapper-pinned versions respected)

Prompts and response parsing live on the language side, not the model side — so every language works with every backend, and teams use whichever AI subscription they already pay for. That's also the cost story: no per-use API fees. Hundreds of runs a day cost the same as one.

What changes for the team

- **Coverage grows with every change** instead of eroding — the net is built at the moment of change, when context is freshest.
- **Bugs surface in minutes, not weeks** — at the cheap end of the cost curve, before they reach QA or customers.
- **Engineer time shifts up a level** — from writing boilerplate checks to reviewing behavior and edge cases.
- **Verified, not vibes** — every generated test is compiled and executed by the real test runner before it's declared done.
- **Human in the loop by default** — tests land uncommitted for review; committing, pushing and PR creation are opt-in.
- **No new budget line** — it runs on the team's existing AI subscriptions, not pay-per-token APIs.

Getting started

```
pip install "git+https://github.com/gautam-calfus/test-automator.git@v0.1"
cd your-project/
test-automator --base-branch main --source-root src
```

Add `--llm copilot` or `--llm gemini` to use a different AI backend, and `--open-pr` to go from diff to reviewed pull request in one command.